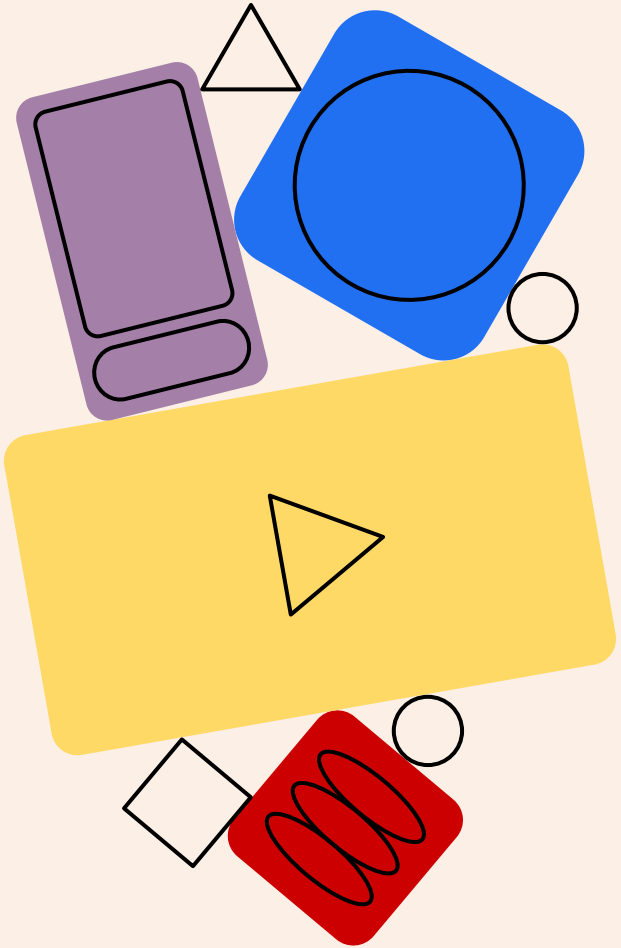
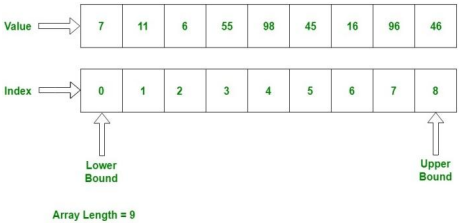
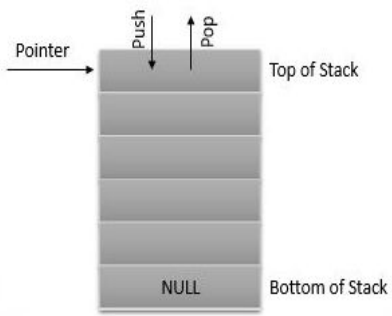
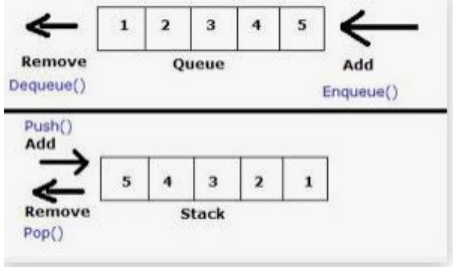
Data Structures

Linear Data Structures - QUEUE

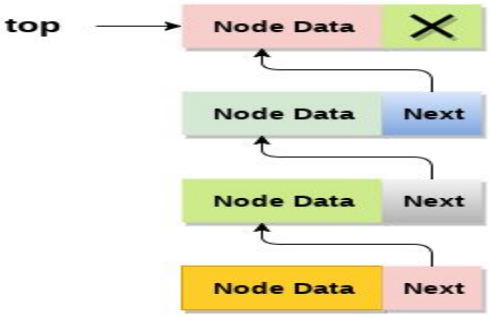


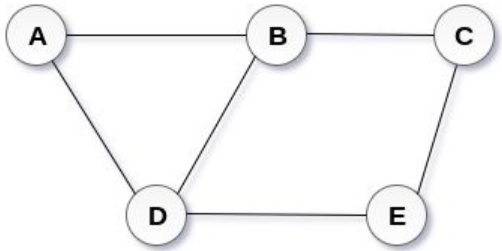

Value
7
11
6
55
98
45
16
96
46
Index
0
1
2
3
4
5
6
7
8
Lower Bound
Upper Bound
Array Length = 9

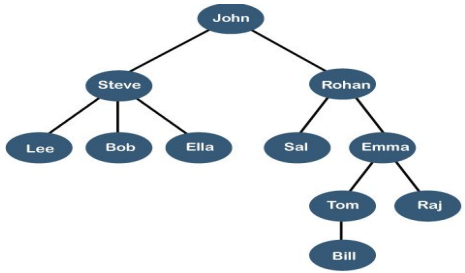

Pointer
Push
Pop
Top of Stack
NULL
Bottom of Stack


1
2
3
4
5
Remove
Dequeue()
Queue
Add
Enqueue()
Push()
Add
5
4
3
2
1
Remove
Pop()
Stack

QUEUE


top
Node Data
Node Data
Next
Node Data
Next
Node Data
Next


A
B
C
D
E


John
Steve
Rohan
Lee
Bob
Ella
Sal
Emma
Tom
Raj
Bill

Introduction

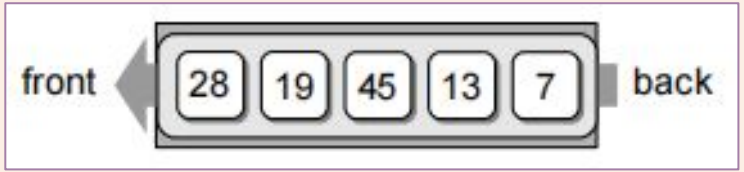
- The term queue is commonly defined to be a line of people waiting to be served like those you would encounter at many business establishments.
- Each person is served based on their position within the queue.
- Thus, the next person to be served is the first in line. As more people arrive, they enter the queue at the back and wait their turn.
- A queue structure is well suited for problems in computer science that require data to be processed in the order in which it was received.
- Some common examples include computer simulations, CPU process scheduling, and shared printer management.

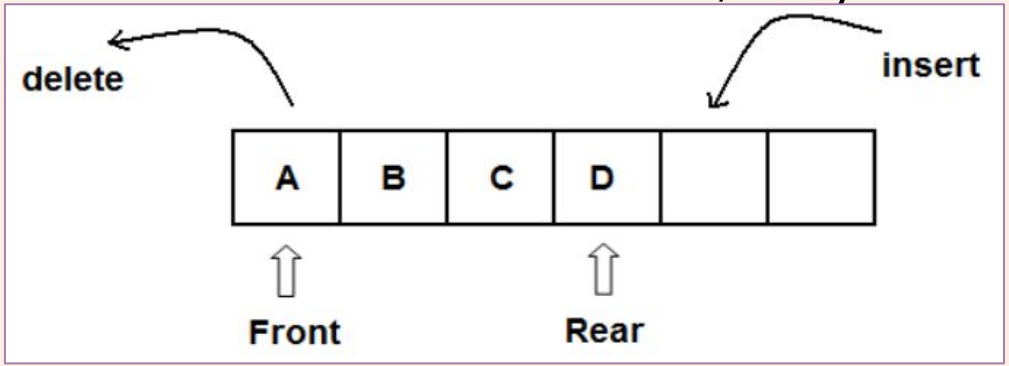
Introduction

- You are familiar with a printer queue if you have used a shared printer.
- Many people may want to use the printer, but only one thing can be printed at a time.
- Instead of making people wait until the printer is not being used to print their document, multiple documents can be submitted at the same time.
- When a document arrives, it is added to the end of the print queue. As the printer becomes available, the document at the front of the queue is removed and printed.
- It is a close “cousin” of the stack, as a queue is a collection of objects that are inserted and removed according to the first-in, first-out (FIFO) principle.

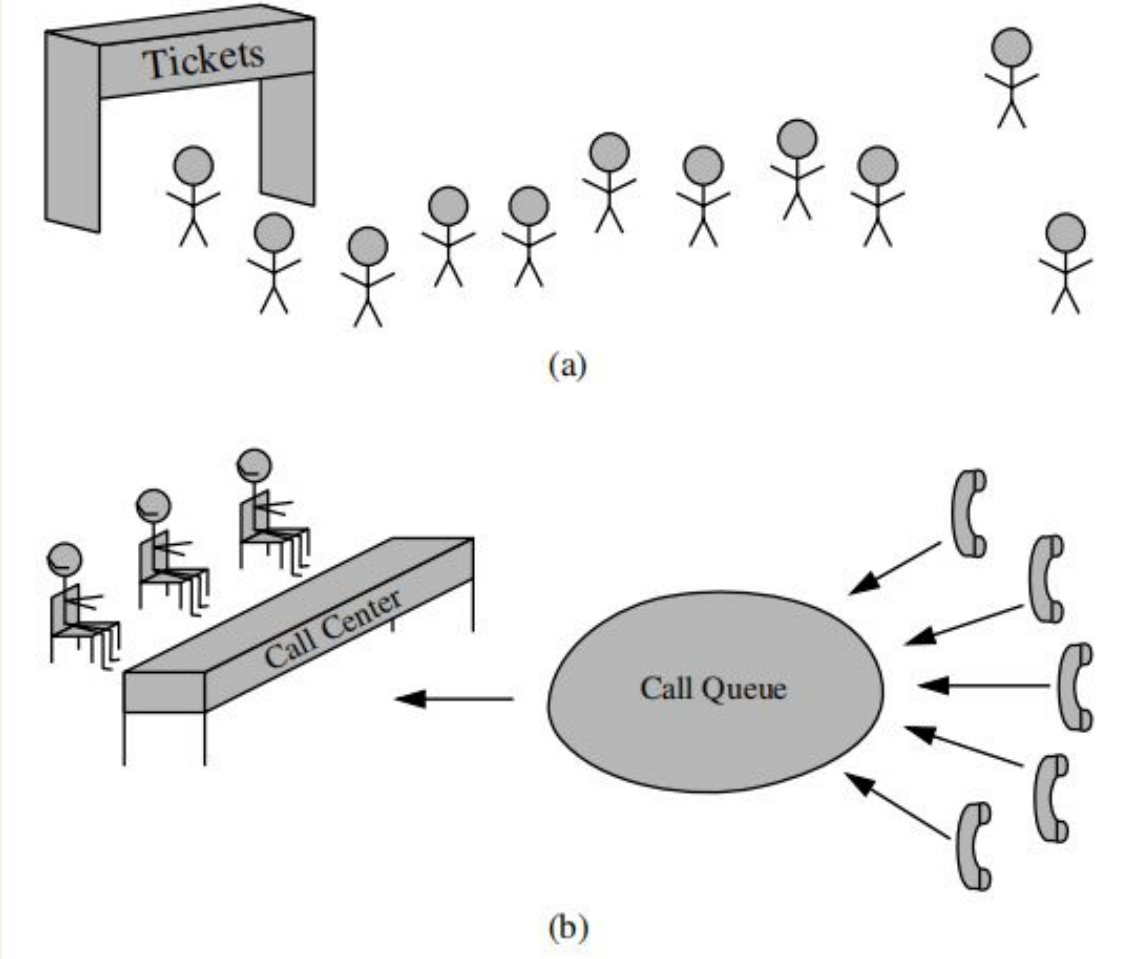
Introduction

- A queue is a specialized list with a limited number of operations in which items can only be added to one end and removed from the other.
- A queue is also known as a first-in, first-out (FIFO) list.
- New items are inserted into a queue at the back while existing items are removed from the front.
- Even though the illustration shows the individual items, they cannot be accessed directly.





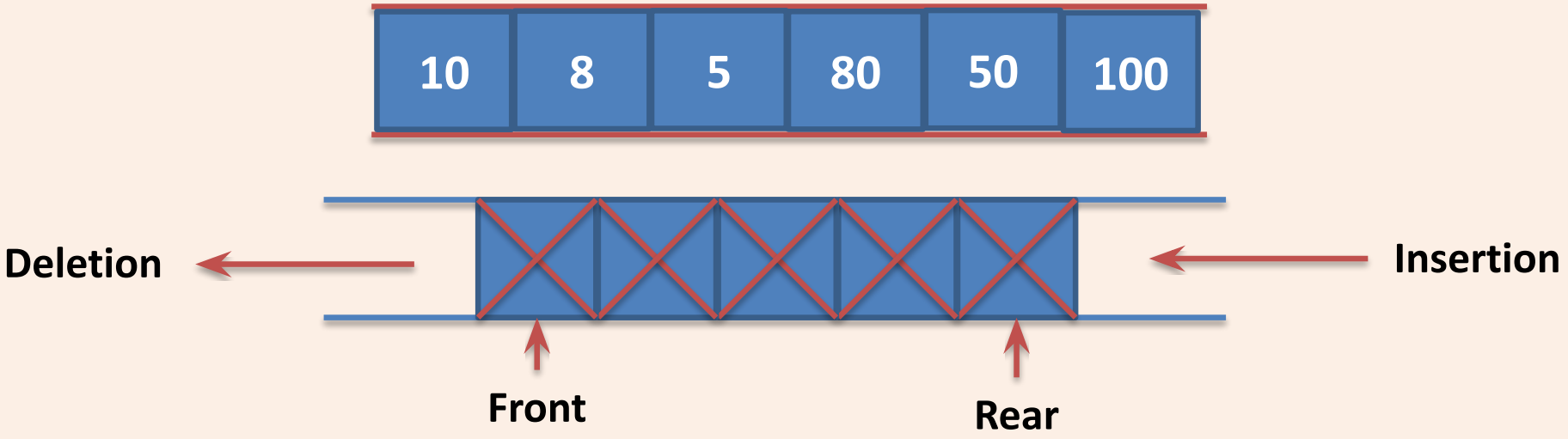
Introduction



Introduction

- A linear list which permits **deletion** to be performed **at one** end of the list and **insertion at the other end** is called **queue**.
- The information in such a list is processed **FIFO (first in first out)** or **FCFS (first come first served)** manner.
- **Front** is the end of queue from that deletion is to be performed.
- **Rear** is the end of queue at which new element is to be inserted.
- Insertion operation is called **Enqueue** and deletion operation is called **Dequeue**.
- Formally, the queue abstract data type defines a collection that keeps objects in a sequence, where element access and deletion are restricted to the first element in the queue, and element insertion is restricted to the back of the sequence.

Introduction



ADT

- The queue abstract data type (ADT) supports the following two fundamental methods for a queue **Q**:
  - **Q.enqueue(e):** Add element e to the back of queue Q.
  - **Q.dequeue( ):** Remove and return the first element from queue Q; an error occurs if the queue is empty.
- The queue ADT may also includes the following supporting methods
  - **Q.first( ):** Return a reference to the element at the front of queue Q, without removing it; an error occurs if the queue is empty.
  - **Q.is_empty( ):** Return True if queue Q does not contain any elements.
  - **len(Q):** Return the number of elements in queue Q; in Python,
- By convention, we assume that a newly created queue is empty.

Illustration

- The table shows a series of queue operations and their effects on an initially empty queue Q of integers.

Operation	Return Value	first ← Q ← last
Q.enqueue(5)	–	[5]
Q.enqueue(3)	–	[5, 3]
len(Q)	2	[5, 3]
Q.dequeue( )	5	[3]
Q.is_empty( )	False	[3]
Q.dequeue( )	3	[ ]
Q.is_empty( )	True	[ ]
Q.dequeue( )	"error"	[ ]
Q.enqueue(7)	–	[7]
Q.enqueue(9)	–	[7, 9]
Q.first( )	7	[7, 9]
Q.enqueue(4)	–	[7, 9, 4]
len(Q)	3	[7, 9, 4]
Q.dequeue( )	7	[9, 4]

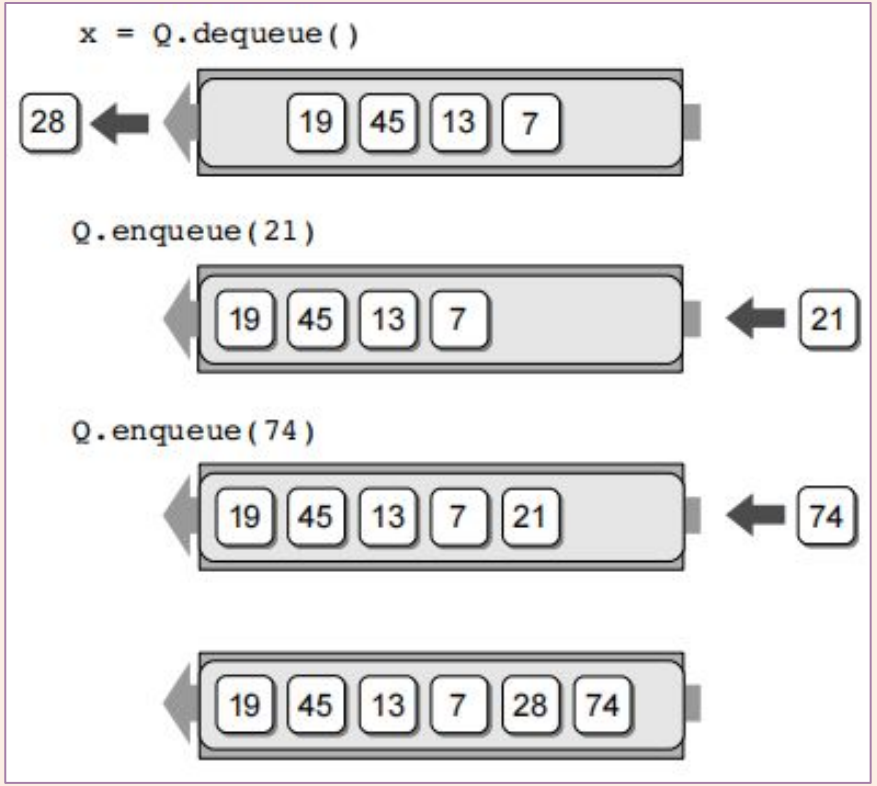
Applications

- Queue of people at any service point such as ticketing etc.
- Queue of air planes waiting for landing instructions.
- Queue of processes in OS.
- Queue is also used by Operating systems for Job Scheduling.
- When a resource is shared among multiple consumers. E.g., in case of printers the first one to be entered is the first to be processed.
- When data is transferred asynchronously (data not necessarily received at same rate as sent) between two processes. Examples include IO Buffers, pipes, file IO, etc.
- Queue is used in BFS (Breadth First Search) algorithm. It helps in traversing a tree or graph.
- Queue is used in networking to handle congestion.
- etc…

Operations - (General)

- EnQueue
- DeQueue
- First/Front
- IS_EMPTY
- IS_FULL

- DISPLAY
- Size
- Resize - Upsize/Downsize
- Length etc.



Procedure: Enqueue (Q, F, R, N,Y)

- This procedure inserts **Y** at rear end of Queue.
- **Queue** is represented by a vector **Q** containing **N** elements.
- **F** is pointer to the front element of a queue.
- **R** is pointer to the rear element of a queue.

```
1. [Check for Queue Overflow]
    If  R >= N
    Then    write ('Queue Overflow')
        Return
2. [Increment REAR pointer]
    R □ R + 1
3. [Insert element]
    Q[R] □ Y
4. [Is front pointer properly set?]
       IF   F=0
       Then F □ 1
       Return
```

Procedure: Enqueue (Q, F, R, N,Y)

```
1. [Check for Queue Overflow]
    If  R >= N
    Then    write ('Queue Overflow')
        Return
2. [Increment REAR pointer]
    R □ R + 1
3. [Insert element]
    Q[R] □ Y
4. [Is front pointer properly set?]
        IF  F=0
        Then    F □ 1
        Return
```

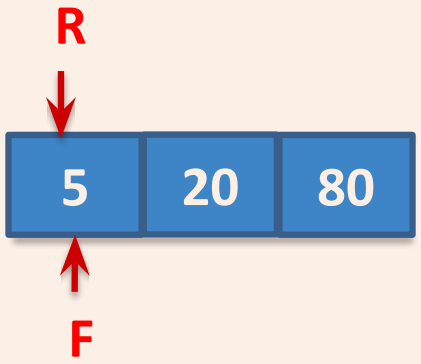
N=3, R=0, F=0

F = 0

R = 0

Enqueue (Q, F, R, N=3,Y=5)
Enqueue (Q, F, R, N=3,Y=20)
Enqueue (Q, F, R, N=3,Y=80)
Enqueue (Q, F, R, N=3,Y=3)

Queue Overflow



Procedure: Dequeue (Q, F, R)

- This function **deletes & returns** an element **from front end** of the Queue.
- **Queue** is represented by a vector **Q** containing **N** elements.
- **F** is pointer to the **front** element of a queue.
- **R** is pointer to the **rear** element of a queue.

```
1. [Check for Queue Underflow]
    If   F = 0
    Then write ('Queue Underflow')
      Return(0)
2. [Delete element]
    Y □ Q[F]
```

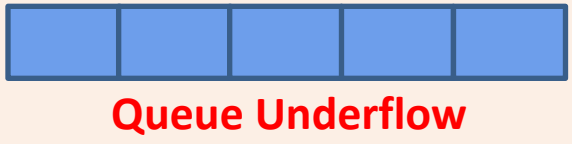
```
3. [Is Queue Empty?]
    If   F = R
    Then F □ R □ 0
    Else F □ F + 1
4. [Return Element]
    Return (Y)
```

Procedure: Dequeue (Q, F, R)

```
1. [Check for Queue Underflow]
    If    F = 0
    Then write ('Queue Underflow')
      Return(0)
2. [Delete element]
    Y □ Q[F]
3. [Is Queue Empty?]
    If    F = R
    Then F □ R □ 0
    Else F □ F + 1
4. [Return Element]
    Return (Y)
```

Case No 1:

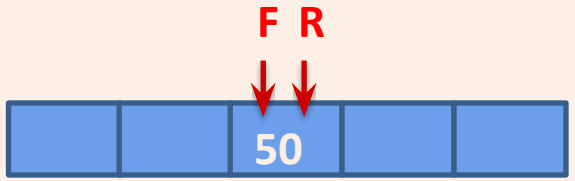
F=0, R=0



Case No 2:

F=3, R=3

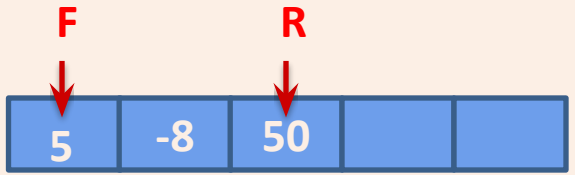
F=0, R=0



Case No 3:

F=1, R=3

F=2, R=3



Example of Queue Insert / Delete

Perform following operations on queue with size 4 & draw queue after each operation
Insert ‘A’ | Insert ‘B’ | Insert ‘C’ | Delete ‘A’ | Delete ‘B’ | Insert ‘D’ | Insert ‘E’

Empty Queue

0 0

F R

Insert ‘A’

R=1
F=1

F R

Insert ‘B’

R=2
F=1

F R

Insert ‘C’

R=3
F=1

F R

Delete ‘A’

R=3
F=2

F R

Delete ‘B’

R=3
F=3

F R

Insert ‘D’

R=4
F=3

F R

Insert ‘E’

R=4
F=3

F R

(R=4) >= (N=4) (Size of Queue)
Queue Overflow

Queue Overflow, but space is there with Queue, this leads to the memory wastage

Implementation in Python

- Array
  - List Class
  - Array Class from Array Module
  - Advanced (Not Discussed)
- Linked List

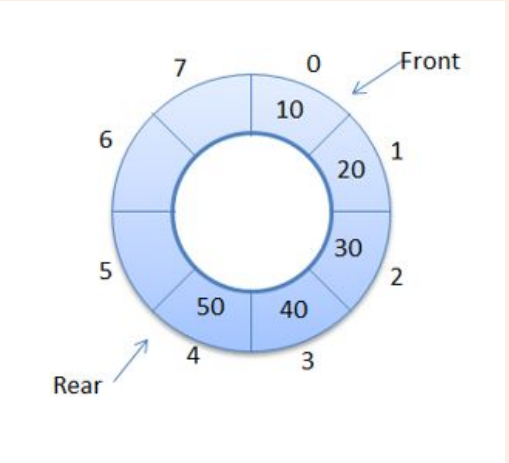
Implementation in Python

- We can implement a queue quite easily by storing its elements in a Python list.
- Fixed Sized Array Implementation.
  or
- Array Implementation using natural python List Size
- Modular / Class Implementation

Circular Queue

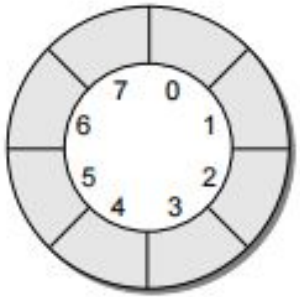
- A more suitable method of representing simple queue which prevents an excessive use of memory is to arrange the elements Q[1], Q[2]….,Q[n] in a circular fashion with Q[1] following Q[n], this is called circular queue.
- In circular queue the last node is connected back to the first node to make a circle.
- Circular queue is a linear data structure. It follows FIFO principle.
- It is also called as “Ring buffer”.

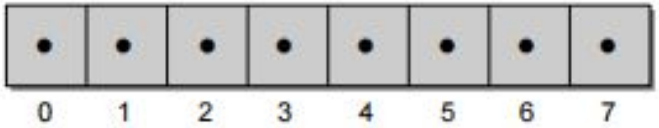
- The list-based implementation of the Queue ADT is easy to implement, but it requires linear time for the enqueue and dequeue operations.
- We can improve these times using an array structure and treating it as a circular array.
- A circular array is simply an array viewed as a circle instead of a line.



Circular Queue

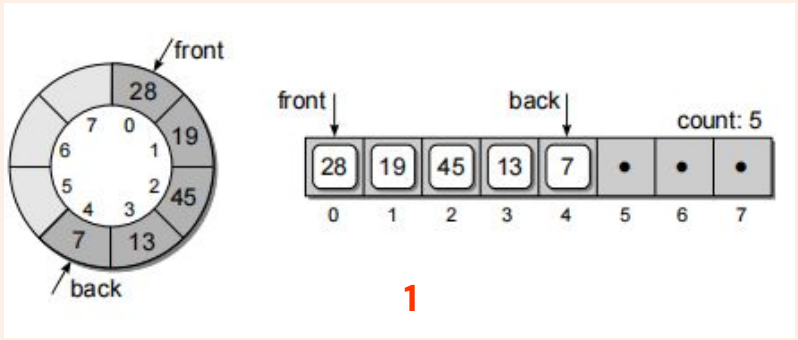
- A circular array allows us to add new items to a queue and remove existing ones without having to shift items in the process.
- Unfortunately, this approach introduces the concept of a maximum-capacity queue that can become full.
- A circular array queue implementation is typically used with applications that only require small-capacity queues and allows for the specification of a maximum size.

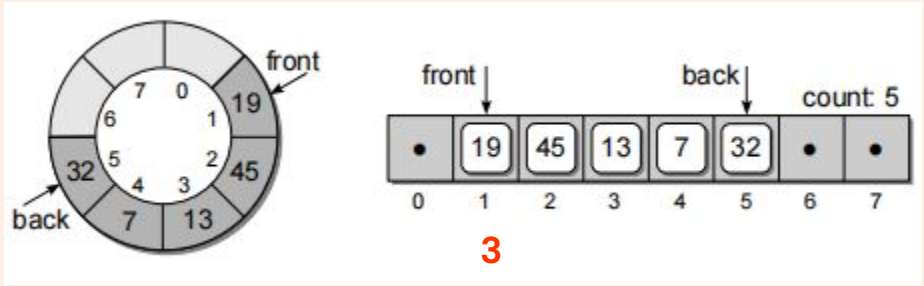


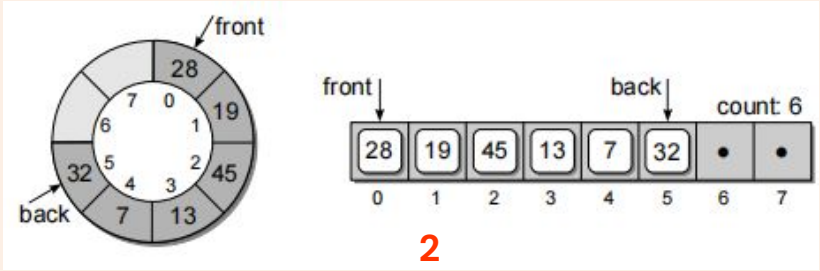


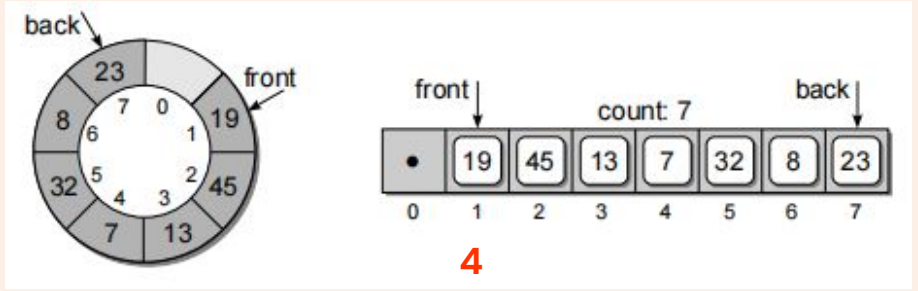
The abstract view of a circular array (left) and the physical view (right).

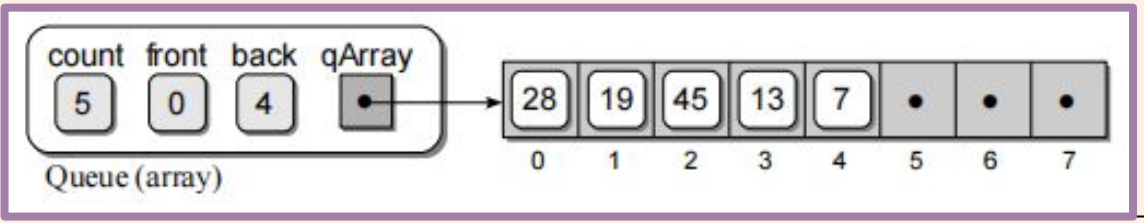
Circular Queue

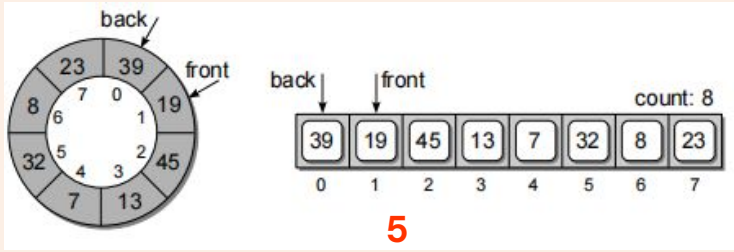






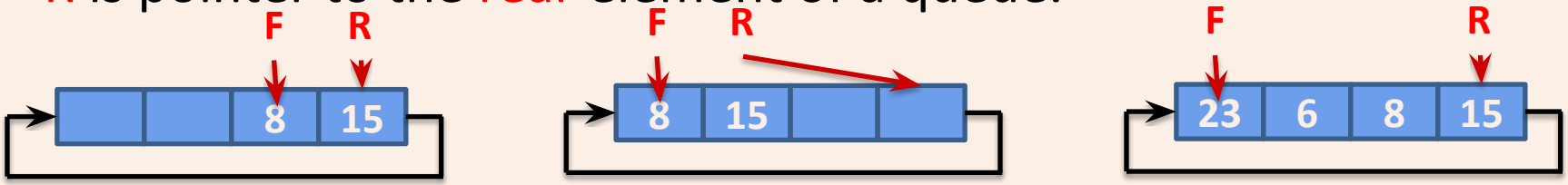






Procedure: CQINSERT (F, R, Q, N, Y)

- This procedure inserts Y at rear end of the Circular Queue.
- Queue is represented by a vector Q containing N elements.
- F is pointer to the front element of a queue.
- R is pointer to the rear element of a queue.




```
1. [Reset Rear Pointer]
   If      R = N
   Then  R □  1
   Else  R □ R + 1
2. [Overflow]
   If    F=R
   Then    Write('Overflow')
      Return
```

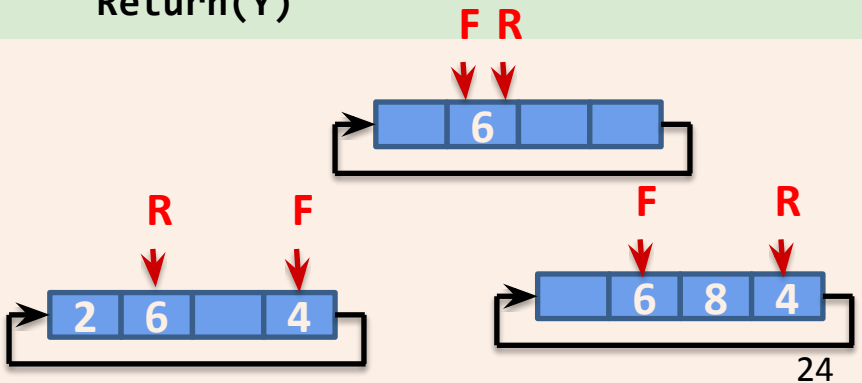
```
3. [Insert element]
    Q[R] □ Y
4. [Is front pointer
   properly set?]
      IF    F=0
      Then  F □ 1

      Return
```

Procedure: CQDELETE (F, R, Q, N)

- This function deletes & returns an element from front end of the Circular Queue.
- Queue is represented by a vector Q containing N elements.
- F is pointer to the front element of a queue.
- R is pointer to the rear element of a queue.

```
1. [Underflow?]
   If      F = 0
   Then  Write('Underflow')
         Return(0)
2. [Delete Element]
    Y □ Q[F]
3. [Queue Empty?]
   If      F = R
   Then  F □ R □  0
         Return(Y)
```

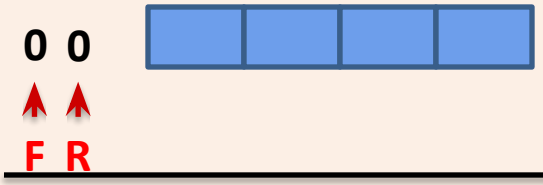
```
4. Increment Front Pointer]
    IF F = N
    Then F □ 1
    Else F □ F + 1
    Return(Y)
```



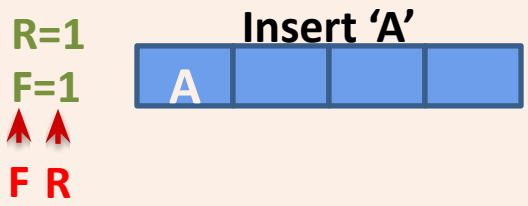
Example of CQueue Insert / Delete

Perform following operations on Circular queue with size 4 & draw queue after each operation **Insert 'A' | Insert 'B' | Insert 'C' | Delete 'A' | Delete 'B' | Insert 'D' | Insert 'E'**

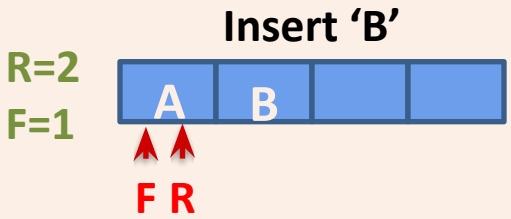
Empty Queue



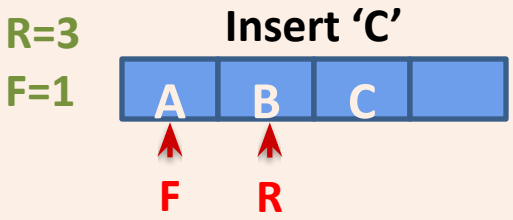
Insert 'A'



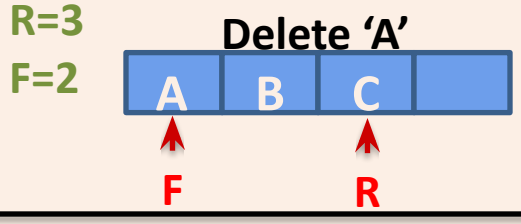
Insert 'B'



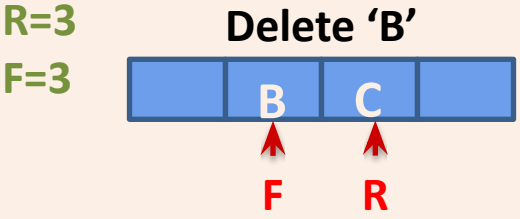
Insert 'C'



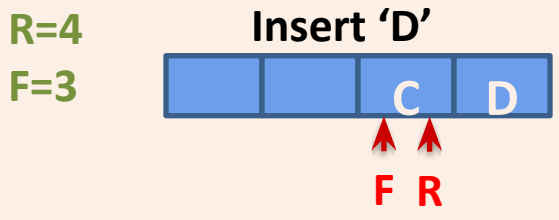
Delete 'A'



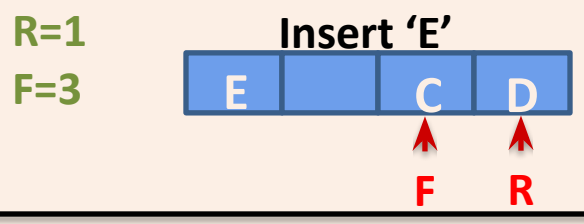
Delete 'B'

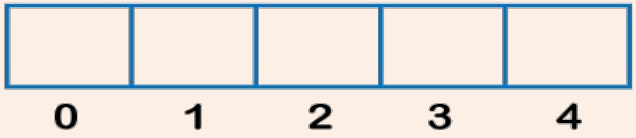


Insert 'D'

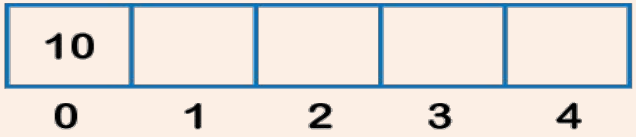


Insert 'E'

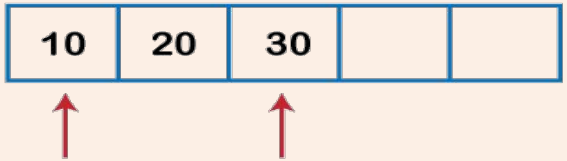


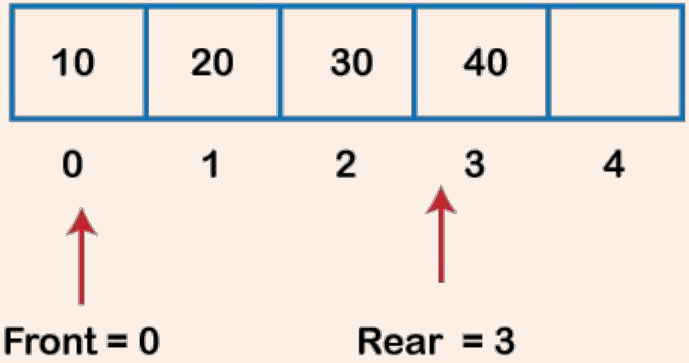


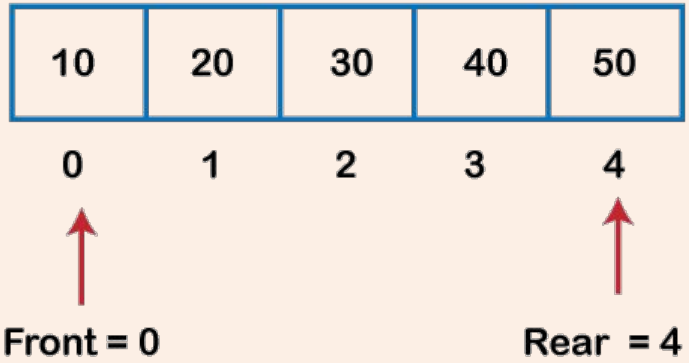
Front = -1
Rear = -1

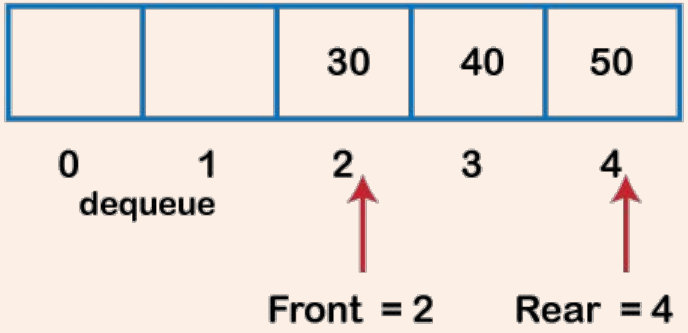


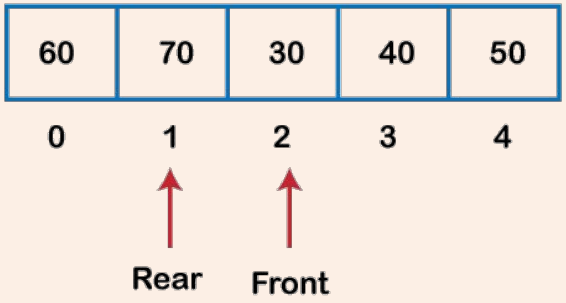
Front = 0
Rear = 0

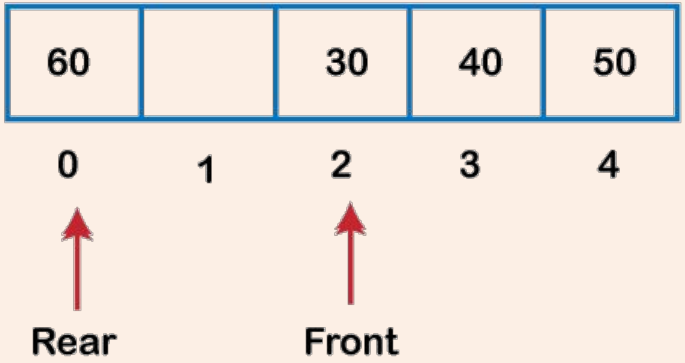






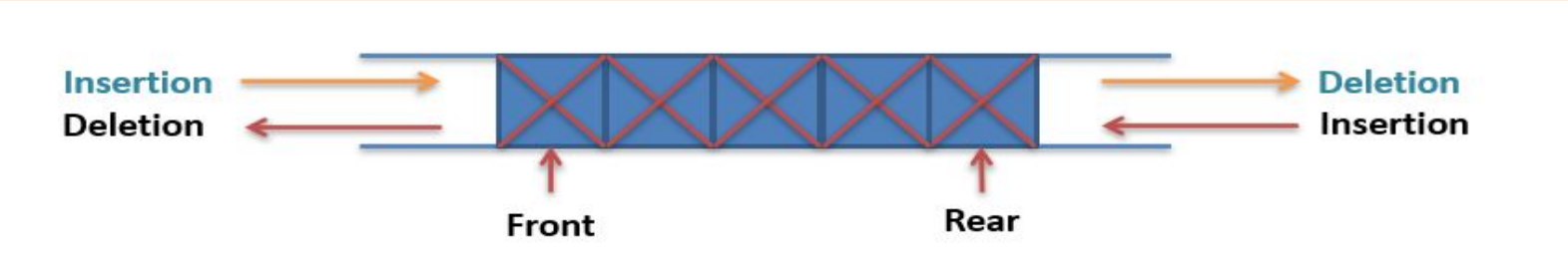

30
40
50
0
1
2
3
4
dequeue
Front = 2
Rear = 4

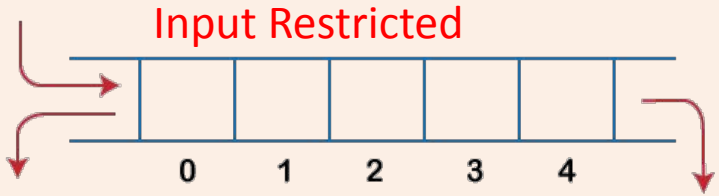

60
70
30
40
50
0
1
2
3
4
Rear
Front

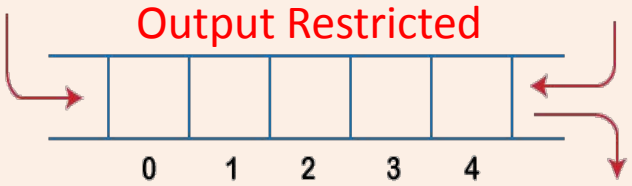

60
30
40
50
0
1
2
3
4
Rear
Front

DQueue - Double Ended Queue

- A **DQueue (double ended queue)** is a linear list in which insertion and deletion are performed **from the either end of the structure**.
- There are two variations of Dqueue
  - ***Input restricted dqueue***- allows insertion at only one end
  - ***Output restricted dqueue***- allows deletion from only one end







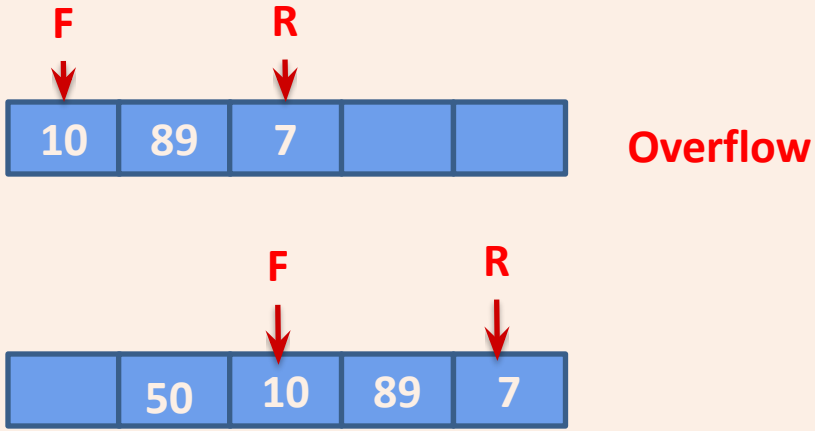
Dqueue Algorithms

- DQINSERT_REAR is same as QINSERT (Enqueue)
- DQDELETE_FRONT is same as QDELETE (Dequeue)
- DQINSERT_FRONT
- DQDELETE_REAR

Procedure: DQINSERT_FRONT (Q,F,R,N,Y)

- This procedure inserts Y at front end of the Queue.
- Queue is represented by a vector Q containing N elements.
- F is pointer to the front element of a queue.
- R is pointer to the rear element of a queue.

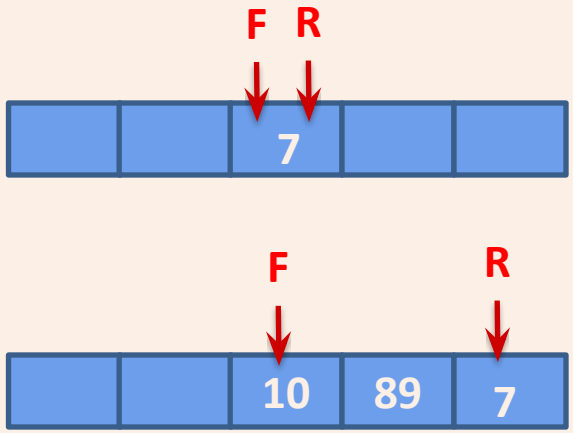
```
1. [Overflow?]
   If      F = 0
   Then  Write('Empty')
         Return
   If      F = 1
   Then  Write('Overflow')
         Return
2. [Decrement front Pointer]
    F □ F - 1
3. [Insert Element?]
   Q[F] □ Y
   Return
```



Procedure: DQDELETE_REAR(Q,F,R)

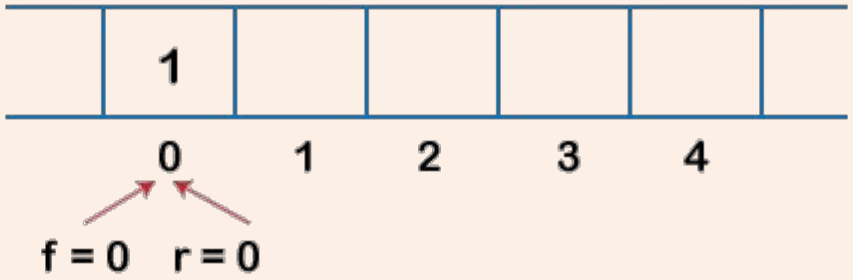
- This function deletes & returns an element from rear end of the Queue.
- Queue is represented by a vector Q containing N elements.
- F is pointer to the front element of a queue.
- R is pointer to the rear element of a queue.

```
1. [Underflow?]
   If       R = 0
   Then  Write('Underflow')
         Return(0)
2. [Delete Element]
    Y □ Q[R]
3. [Queue Empty?]
   IF   R = F
   Then R □ F □ 0
   Else R □ R - 1
4. [Return Element]
   Return(Y)
```



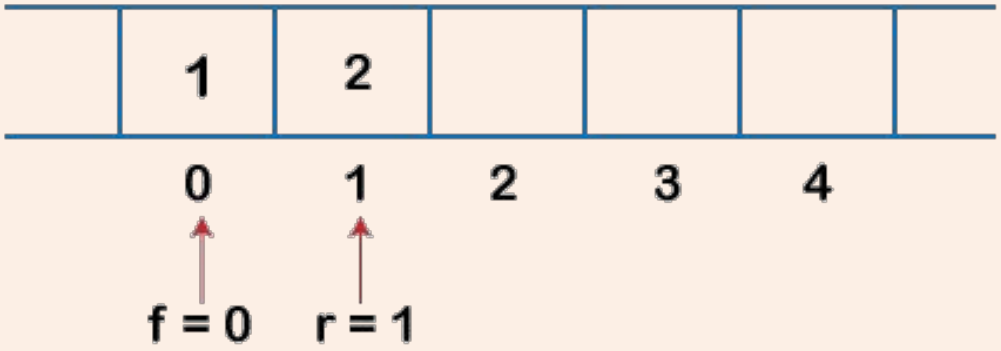
Implementation of Deque using a circular array Enqueue operation

1. Initially, we are considering that the deque is empty, so both front and rear are set to -1, i.e., **f = -1** and **r = -1**.
2. As the deque is empty, so inserting an element either from the front or rear end would be the same thing. Suppose we have inserted element 1, then **front is equal to 0,** and the **rear is also equal to 0.**



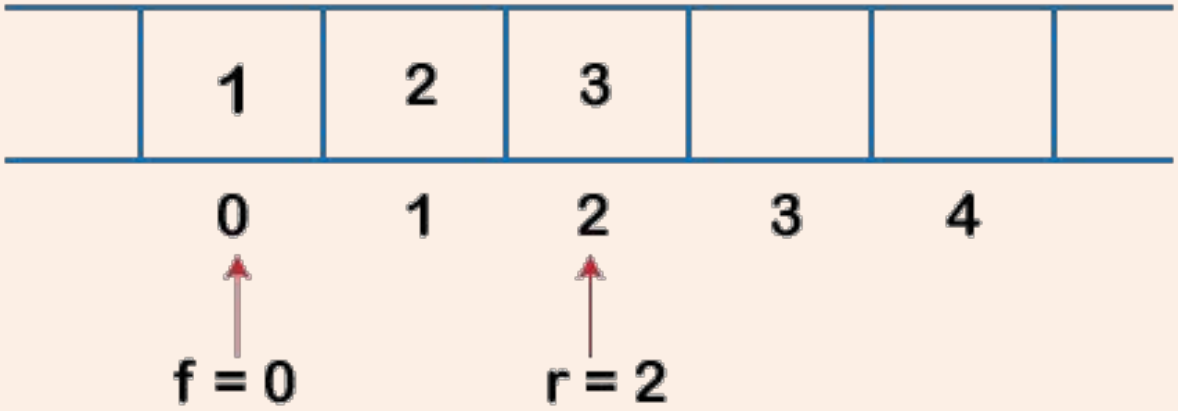
Implementation of Deque using a circular array Enqueue operation

3. Suppose we want to insert the next element from the rear. To insert the element from the rear end, we first need to increment the rear, i.e., **rear=rear+1**. Now, the rear is pointing to the second element, and the front is pointing to the first element.



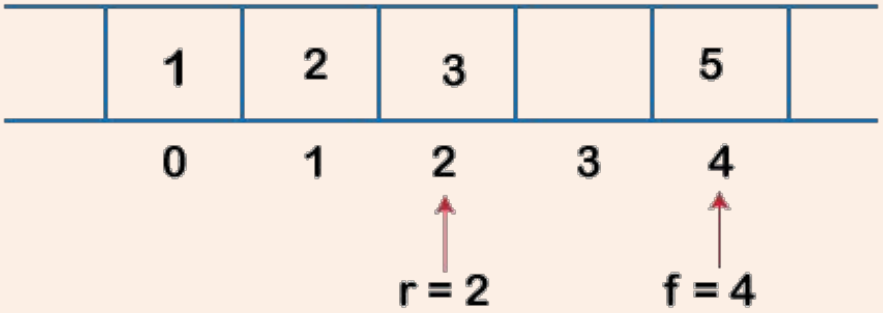
Implementation of Deque using a circular array Enqueue operation

4. Suppose we are again inserting the element from the rear end. To insert the element, we will first increment the rear, and now rear points to the third element.



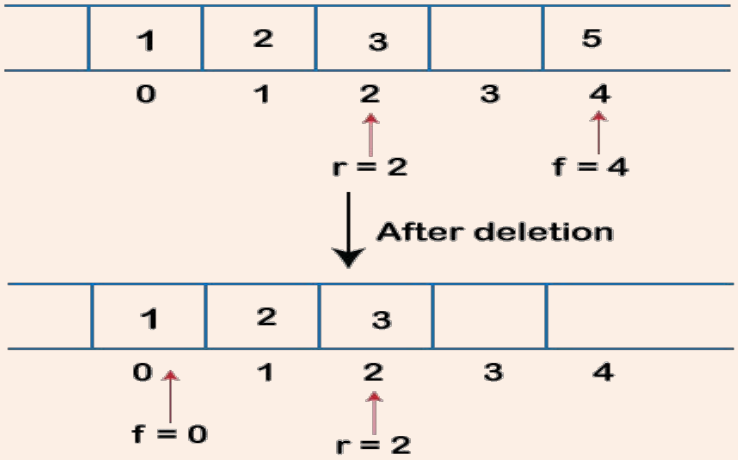
Implementation of Deque using a circular array Enqueue operation

5. If we want to insert the element from the front end, and insert an element from the front, we have to decrement the value of front by 1. If we decrement the front by 1, then the front points to -1 location, which is not any valid location in an array. So, we set the front as **(n -1),** which is equal to 4 as n is 5. Once the front is set, we will insert the value



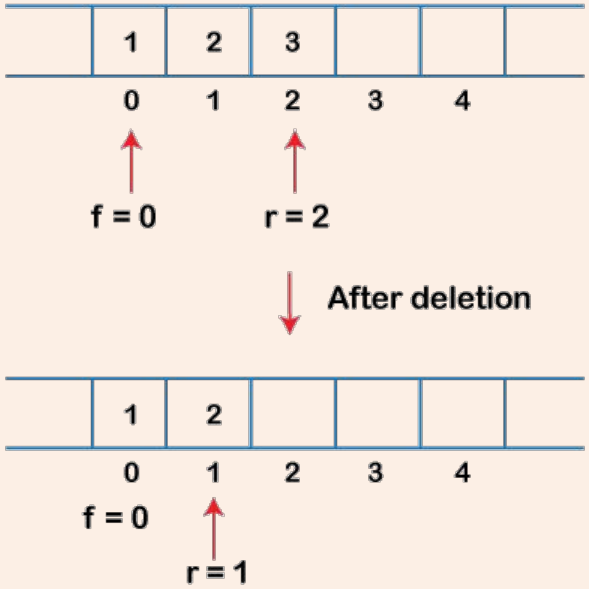
Implementation of Deque using a circular array Dequeue operation

1. If the front is pointing to the last element of the array, and we want to perform the delete operation from the front. To delete any element from the front, we need to set **front=front+1**. Currently, the value of the front is equal to 4, and if we increment the value of front, it becomes 5 which is not a valid index. Therefore, we conclude that if front points to the last element, then front is set to 0 in case of delete operation.



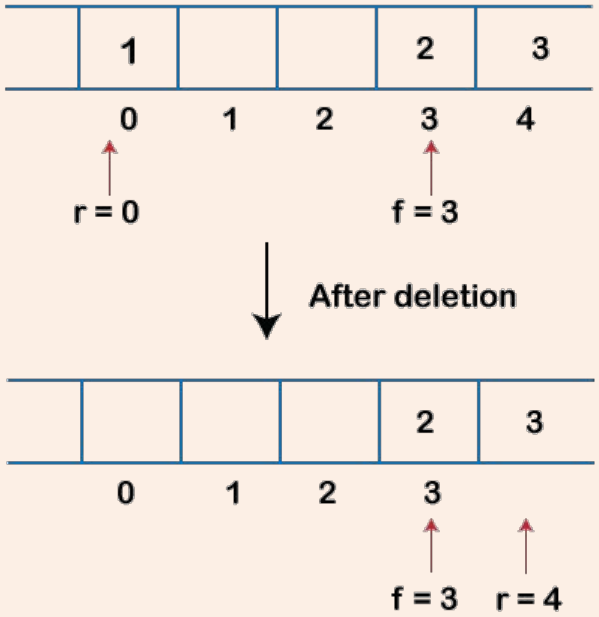
Implementation of Deque using a circular array Dequeue operation

2. If we want to delete the element from rear end then we need to decrement the rear value by 1, i.e., **rear=rear-1**



Implementation of Deque using a circular array Dequeue operation

3. If the rear is pointing to the first element, and we want to delete the element from the rear end then we have to set **rear=n-1** where **n** is the size of the array



Implementation of Deque using a array

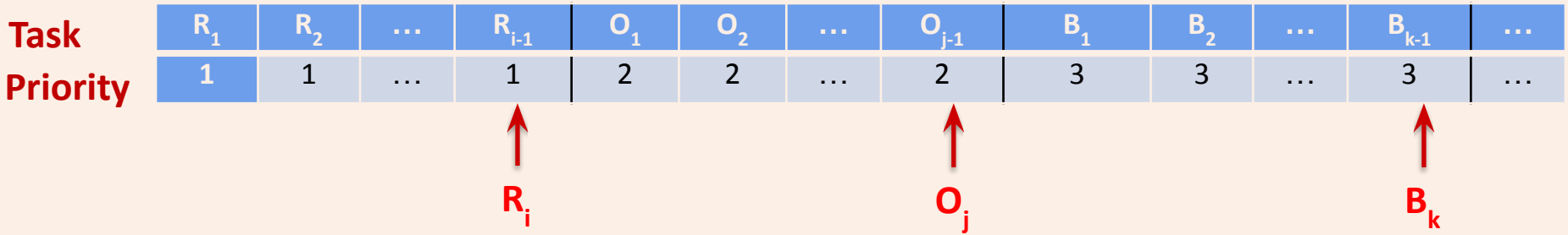
The following are the six functions that we can use in the program:

- **enqueue_front()**: It is used to insert the element from the front end.
- **enqueue_rear()**: It is used to insert the element from the rear end.
- **dequeue_front()**: It is used to delete the element from the front end.
- **dequeue_rear()**: It is used to delete the element from the rear end.
- **getfront()**: It is used to return the front element of the deque.
- **getrear()**: It is used to return the rear element of the deque.

Priority Queue

- A queue in which we are able to **insert & remove items** from **any position based on** some property (such as **priority** of the task to be processed) is often referred as **priority queue**.
- So that it does not support FIFO(First In First Out) structure entirely.
- Priorities are attached with each Job
  - **Priority 1** indicates **Real Time Job**
  - **Priority 2** indicates **Online Job**
  - **Priority 3** indicates **Batch Processing Job**
- Therefore if a job is initiated with priority i, it is inserted immediately at the end of list of other jobs with priorities i.
- Here jobs are always removed from the front of queue

Priority Queue



Priority Queue viewed as a single queue with insertion allowed at any position

Priority Queue viewed as a Viewed as a set of queue

Priority Queue

- A priority queue is a special type of queue in which each element is associated with a priority and is served according to its priority.
- If elements with the same priority occur, they are served according to their order in the queue.
- Generally, the value of the element itself is considered for assigning the priority.
- For example, The element with the highest value is considered as the highest priority element.
- However, in other cases, we can assume the element with the lowest value as the highest priority element.

Difference between Priority Queue and Normal Queue

- In a queue, the **first-in-first-out rule** is implemented whereas, in a priority queue, the values are removed **on the basis of priority**. The element with the highest priority is removed first.

- **Basic Operations**
  - **insert / enqueue −** add an item to the rear of the queue.
  - Whenever an element is inserted into queue, priority queue inserts the item according to its order. Here we're assuming that data with high value has low priority.
  - **remove / dequeue −** remove an item from the front of the queue.

Priority Queue

Types of Priority Queue:

- Min Priority Queue: Minimum number of value gets the highest priority and lowest number of element gets the highest priority.
- Max Priority Queue: Maximum number value gets the highest priority and minimum number of value gets the minimum priority.

Types of Priority Queue:

- Ascending priority queue
  - Removal of minimum-priority element
  - Remove-top(): Removes element with min priority
- Descending priority queue
  - Removal of maximum-priority element
  - Remove-top(): Removes element with max priority

Priority Queue

Priority Queue Approaches:

Priority Queue can be implemented in two ways:

- Using ordered Array: In ordered array enqueue operation takes O(n) time complexity because it enters elements in sorted order in queue. And deletion takes O(1) time complexity.
- Using unordered Array:In unordered array deletion takes O(n) time complexity because it search for the element in Queue for the deletion and enqueue takes o(1) time complexity.

Priority Queue

- A priority queue is a data structure in which each element is assigned a priority.
- The priority of the element will be used to determine the order in which the elements will be processed.
- The general rules of processing the elements of a priority queue are:
  - An element with higher priority is processed before an element with a lower priority.
  - Two elements with the same priority are processed on a first-come-first-served (FCFS) basis.
- A priority queue can be thought of as a modified queue in which when an element has to be removed from the queue, the one with the highest-priority is retrieved first.

Priority Queue

- The priority of the element can be set based on various factors.
- Priority queues are widely used in operating systems to execute the highest priority process first.
- The priority of the process may be set based on the CPU time it requires to get executed completely.
- For example, if there are three processes,
  - where the first process needs 5 ns to complete,
  - the second process needs 4 ns,
  - and the third process needs 7 ns,
- then the second process will have the highest priority and will thus be the first to be executed.

Priority Queue

- However, CPU time is not the only factor that determines the priority, rather it is just one among several factors.
- Another factor is the importance of one process over another.
- In case we have to run two processes at the same time, where one process is concerned with online order booking and the second with printing of stock details, then obviously the online booking is more important and must be executed first.

Priority Queue

- There are two ways to implement a priority queue.
- We can either use a sorted list to store the elements so that when an element has to be taken out, the queue will not have to be searched for the element with the highest priority or we can use an unsorted list so that insertions are always done at the end of the list.
- Every time when an element has to be removed from the list, the element with the highest priority will be searched and removed.
- While a sorted list takes O(n) time to insert an element in the list, it takes only O(1) time to delete an element.
- On the contrary, an unsorted list will take O(1) time to insert an element and O(n) time to delete an element from the list.
- Practically, both these techniques are inefficient and usually a blend of these two approaches is adopted that takes roughly O(log n) time or less.

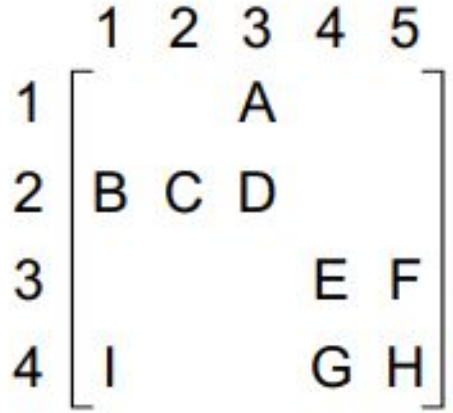
Array Representation of a Priority Queue

- When arrays are used to implement a priority queue, then a separate queue for each priority number is maintained.
- Each of these queues will be implemented using circular arrays or circular queues.
- Every individual queue will have its own FRONT and REAR pointers.
- We use a two-dimensional array for this purpose where each queue will be allocated the same amount of space.
- Look at the two-dimensional representation of a priority queue given below.
- Given the front and rear values of each queue, the two-dimensional matrix can be formed as shown in Fig.

Array Representation of a Priority Queue

- FRONT[K] and REAR[K] contain the front and rear values of row K, where K is the priority number.
- Note that here we are assuming that the row and column indices start from 1, not 0. Obviously, while programming, we will not

FRONT	REAR
3	3
1	3
4	5
4	1



Array Representation of a Priority Queue

Insertion

- To insert a new element with priority K in the priority queue, add the element at the rear end of row K, where K is the row number as well as the priority number of that element.
- For example, if we have to insert an element R with priority number 3, then the priority queue will be given as shown in Fig.

FRONT	REAR
3	3
1	3
4	1
4	1

	1	2	3	4	5
1			A		
2	B	C	D		
3	R			E	F
4	I			G	H

Array Representation of a Priority Queue

- **Deletion**
- To delete an element, we find the first non empty queue and then process the front element of the first non-empty queue.
- In our priority queue, the first non-empty queue is the one with priority number 1 and the front element is A, so A will be deleted and processed first.
- In technical terms, find the element with the smallest K, such that FRONT[K] != NULL.

FRONT	REAR
1	3
4	1
4	1

	1	2	3	4	5
1					
2	B	C	D		
3	R			E	F
4	I			G	H

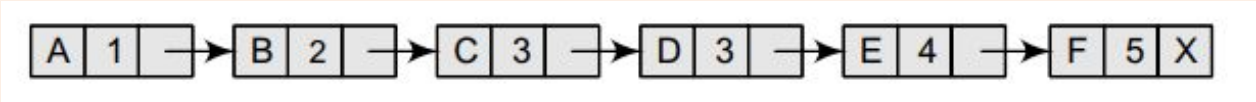
Linked Representation of a Priority Queue

- When a priority queue is implemented using a linked list, then every node of the list will have three parts:
  (a) the information or data part,
  (b) the priority number of the element, and
  (c) the address of the next element.
- If we are using a sorted linked list, then the element with the higher priority will precede the element with the lower priority.
- Consider the priority queue shown in Fig.

- Lower priority number means higher priority.
- For example, if there are two elements A and B, where A has a priority number 1 and B has a priority number 5, then A will be processed before B as it has higher priority than B.

Linked Representation of a Priority Queue

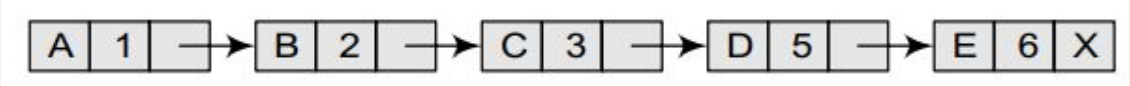
- The priority queue in Fig. is a sorted priority queue having six elements.
- From the queue, we cannot make out whether A was inserted before E or whether E joined the queue before A because the list is not sorted based on FCFS.
- Here, the element with a higher priority comes before the element with a lower priority.
- However, we can definitely say that C was inserted in the queue before D because when two elements have the same priority the elements are arranged and processed on FCFS principle.



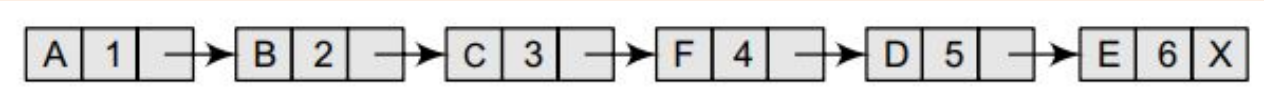
Linked Representation of a Priority Queue

Insertion

- When a new element has to be inserted in a priority queue, we have to traverse the entire list until we find a node that has a priority lower than that of the new element.
- The new node is inserted before the node with the lower priority.
- However, if there exists an element that has the same priority as the new element, the new element is inserted after that element.
- For example, consider the priority queue shown in Fig.



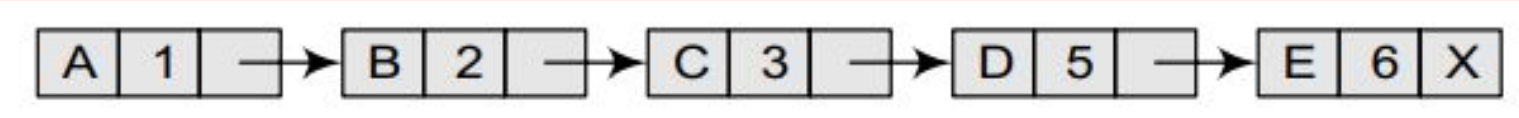
- If we have to insert a new element with data = F and priority number = 4, then the element will be inserted before D that has priority number 5, which is lower priority than that of the new element.

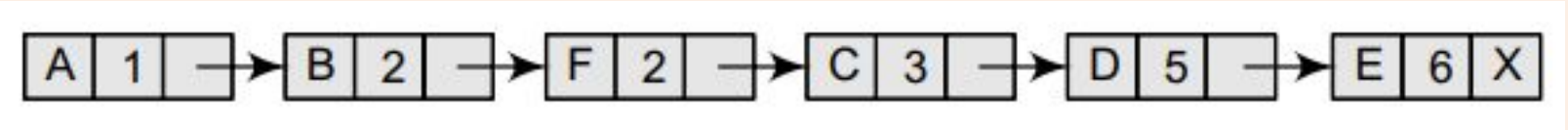


Linked Representation of a Priority Queue

Insertion

- if we have a new element with data = F and priority number = 2, then the element will be inserted after B, as both these elements have the same priority but the insertions are done on FCFS basis as shown in Fig.



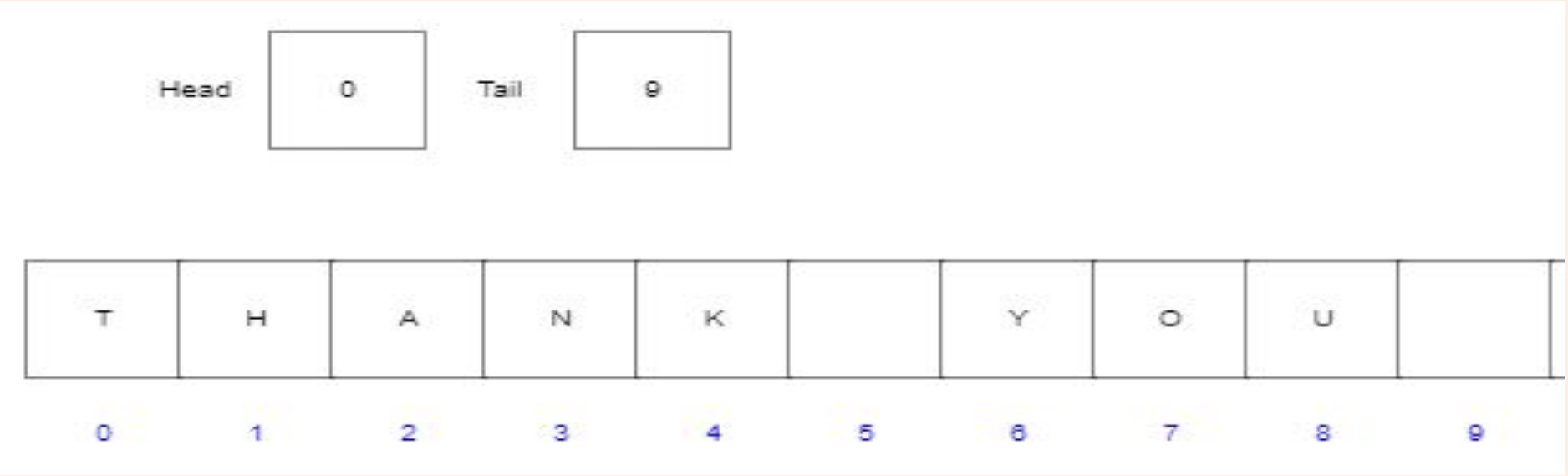


Deletion

- Deletion is a very simple process in this case. The first node of the list will be deleted and the data of that node will be processed first.

The complexity of the operations of a priority queue using arrays, linked list, and the binary heap is given in the table below.

Data Structure	**EnQueue**	**DeQueue**	**Peek**
Ordered Array	O(n)	O(1)	O(1)
Unordered Array	O(1)	O(n)	O(n)
Ordered Linked List	O(n)	O(1)	O(1)
Unordered Linked List	O(1)	O(n)	O(n)
Binary Heap	O(log n)	O(log n)	O(1)


Head
0
Tail
9
T
H
A
N
K
Y
O
U
0
1
2
3
4
5
6
7
8
9


Thank You